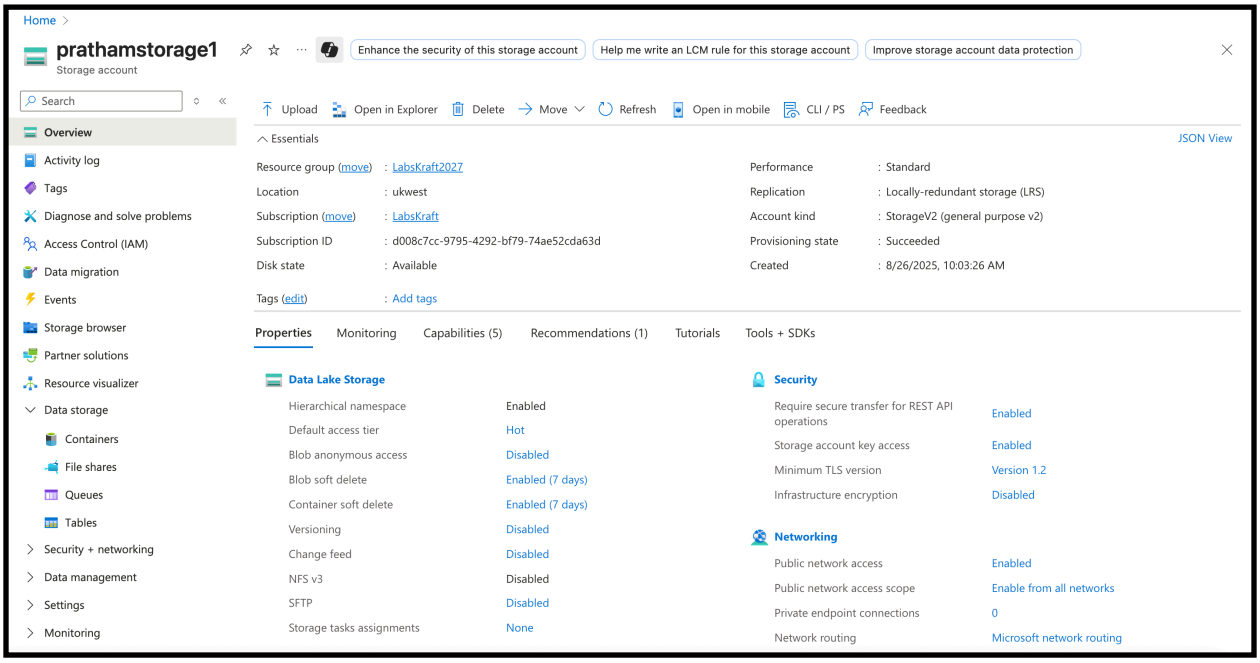
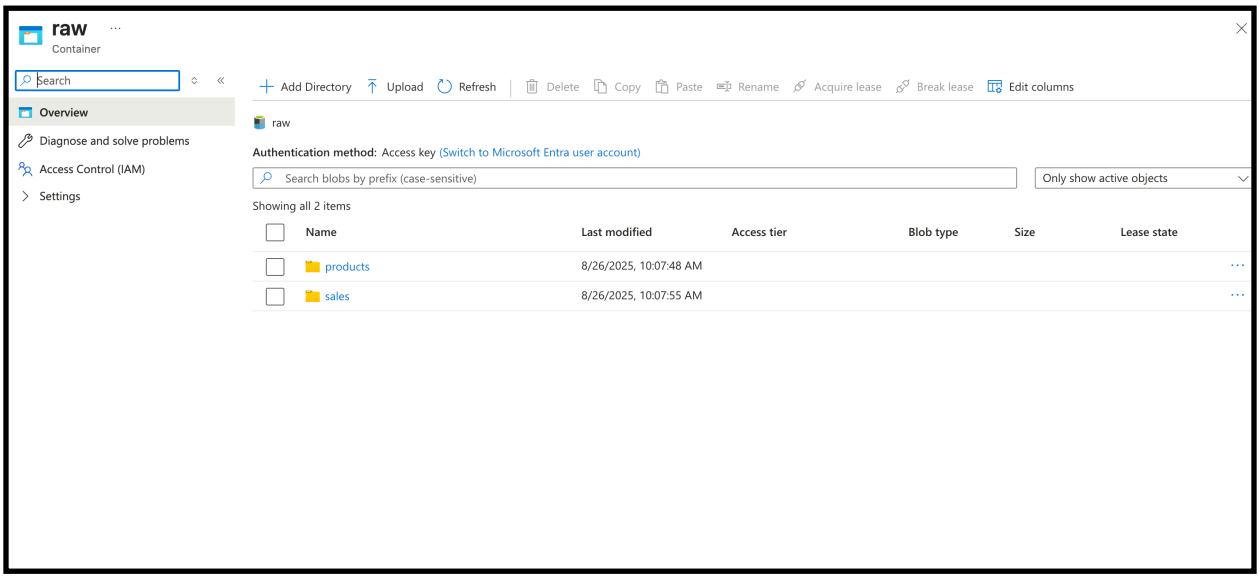
Pratham Bhoot
Capstone Project-2

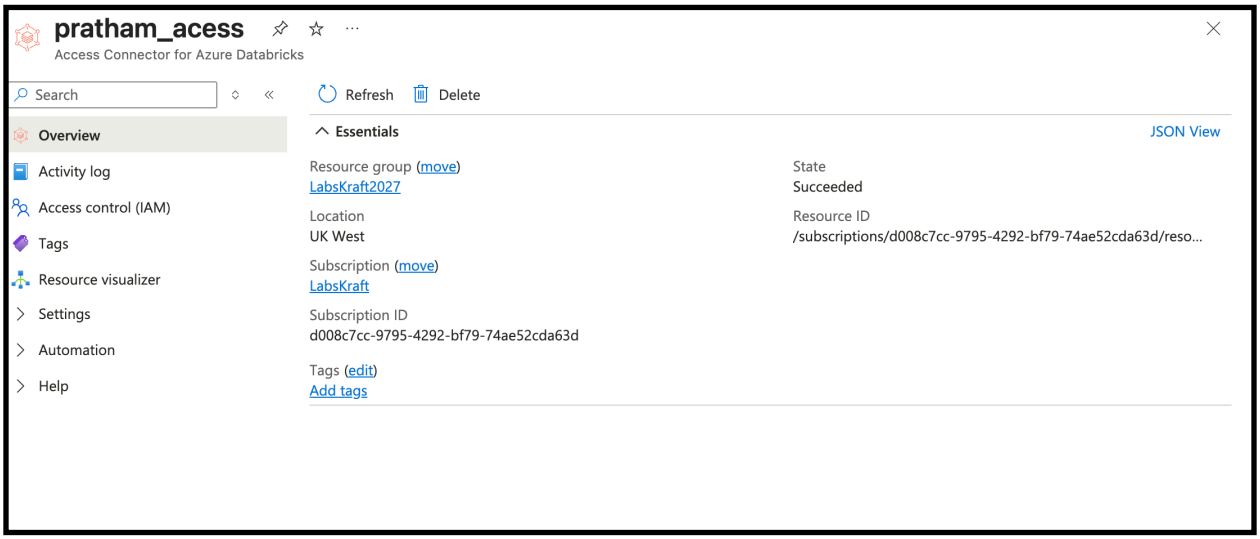
Creating Storage Account:



Creating Container:

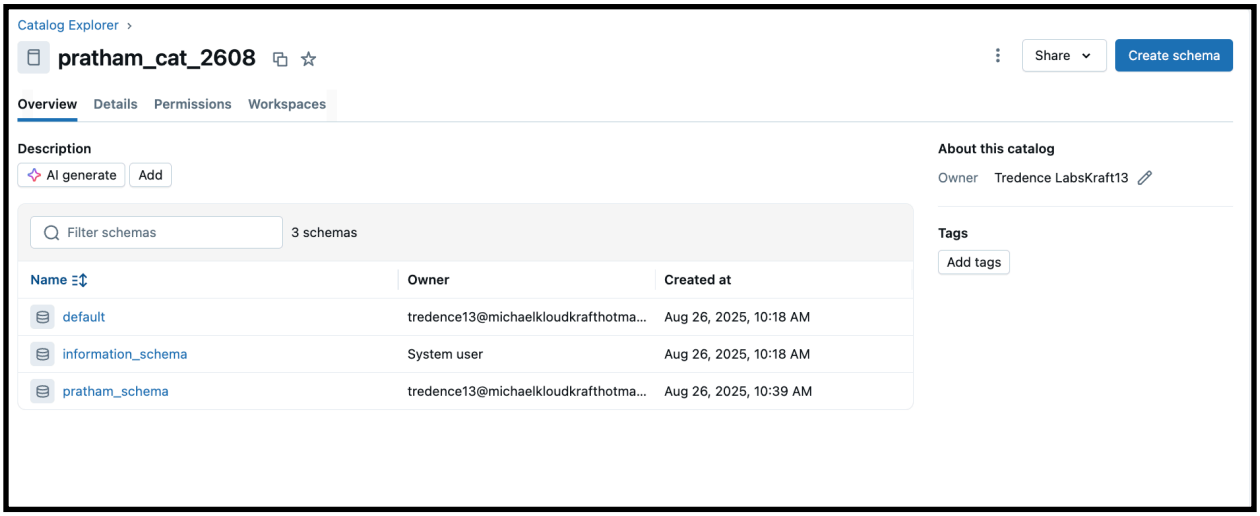


Creating Access Connector:

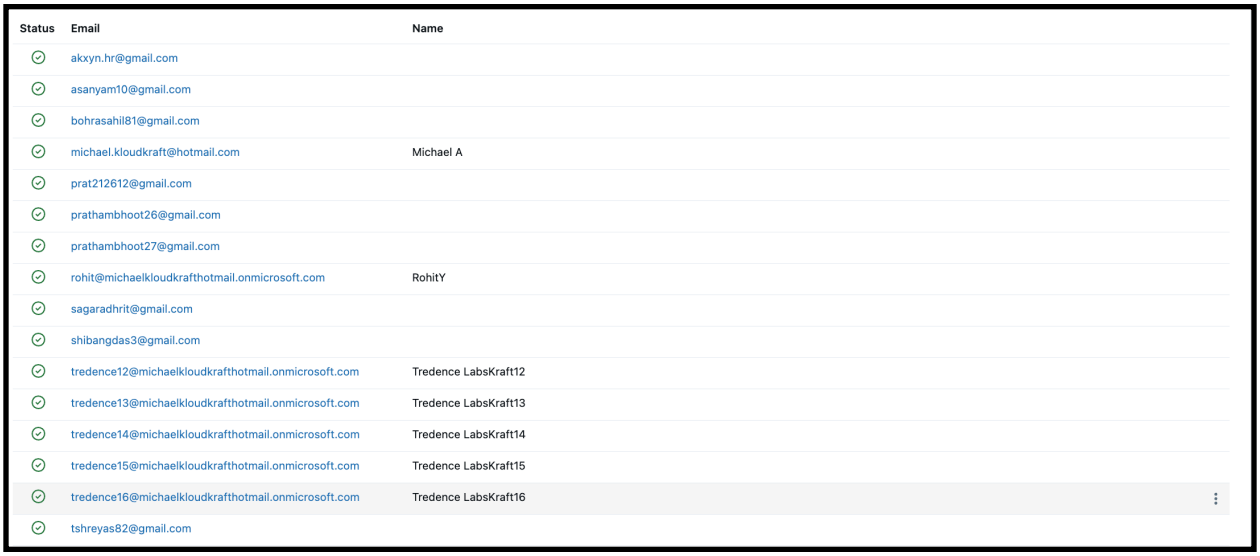


Creating External Location:

Creation of Unity Catalog:

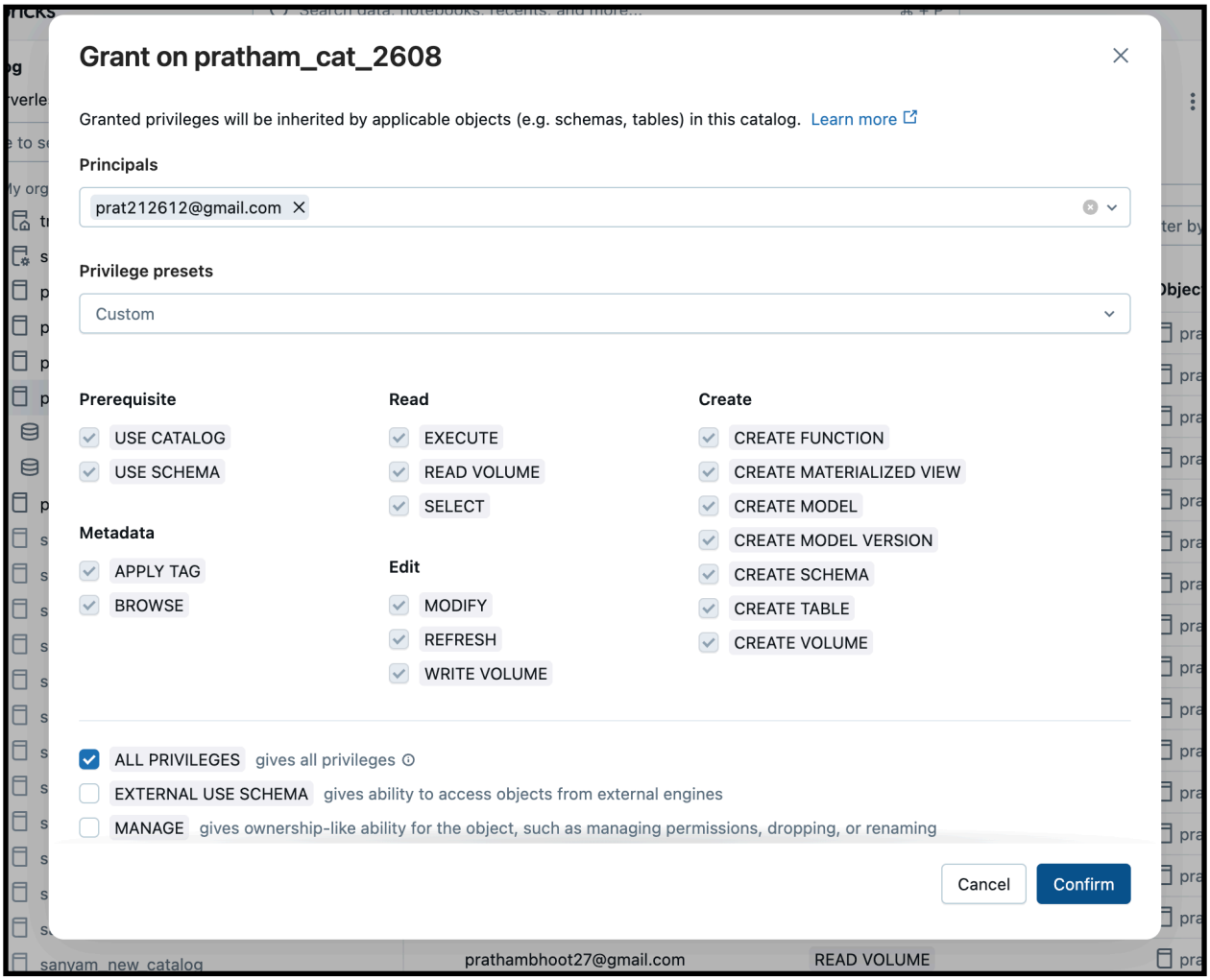


Adding Users to Unity Catalog:



Status	Email	Name
✓	akxyn.hr@gmail.com	
✓	asanyam10@gmail.com	
✓	bohrasahil81@gmail.com	
✓	michael.kloudkraft@hotmail.com	Michael A
✓	prat212612@gmail.com	
✓	prathambhoot26@gmail.com	
✓	prathambhoot27@gmail.com	
✓	rohit@michaelkloudkrafthotmail.onmicrosoft.com	RohitY
✓	sagaradhrit@gmail.com	
✓	shibangdas3@gmail.com	
✓	tredence12@michaelkloudkrafthotmail.onmicrosoft.com	Tredence LabsKraft12
✓	tredence13@michaelkloudkrafthotmail.onmicrosoft.com	Tredence LabsKraft13
✓	tredence14@michaelkloudkrafthotmail.onmicrosoft.com	Tredence LabsKraft14
✓	tredence15@michaelkloudkrafthotmail.onmicrosoft.com	Tredence LabsKraft15
✓	tredence16@michaelkloudkrafthotmail.onmicrosoft.com	Tredence LabsKraft16
✓	tshreyas82@gmail.com	

Granting Access to Users:



Creation of Schema:

```
%sql
USE CATALOG `pratham_cat_2608`;
CREATE SCHEMA IF NOT EXISTS pratham_schema;
```

Creation of Pipeline:

Code:

A) Bronze:

```
import dlt
from pyspark.sql.functions import *

# Ingest raw sales data (multiple daily CSV files in raw/sales/)
@dlt.table(
  name="sales_raw",
  comment="Raw sales data ingested from CSV files",
  table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
        spark.readStream.format("cloudFiles")     # Auto Loader for
incremental loads
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://raw@prathamstorage1.dfs.core.windows.net/sales/")
    )

# Products CSV
@dlt.table(
  name="products_raw_csv",
  comment="Raw product data from CSV",
  table_properties={"quality": "bronze"}
)
def products_raw_csv():
    return (
        spark.readStream.format("cloudFiles")
```

```
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://raw@prathamstorage1.dfs.core.windows.net/
products/")
    )

# Products JSON
@dlt.table(
  name="products_raw_json",
  comment="Raw product data from JSON",
  table_properties={"quality": "bronze"}
)
def products_raw_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .load("abfss://raw@prathamstorage1.dfs.core.windows.net/
products/")
    )

# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
    return
dlt.read("products_raw_csv").unionByName(dlt.read("products_raw_json"),
allowMissingColumns=True)
```

B) Silver:

```
# Clean Sales
@dlt.table(
  name="sales_clean",
  comment="Cleaned sales data with enforced schema",
  table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
    return (
        dlt.read("sales_raw")
```

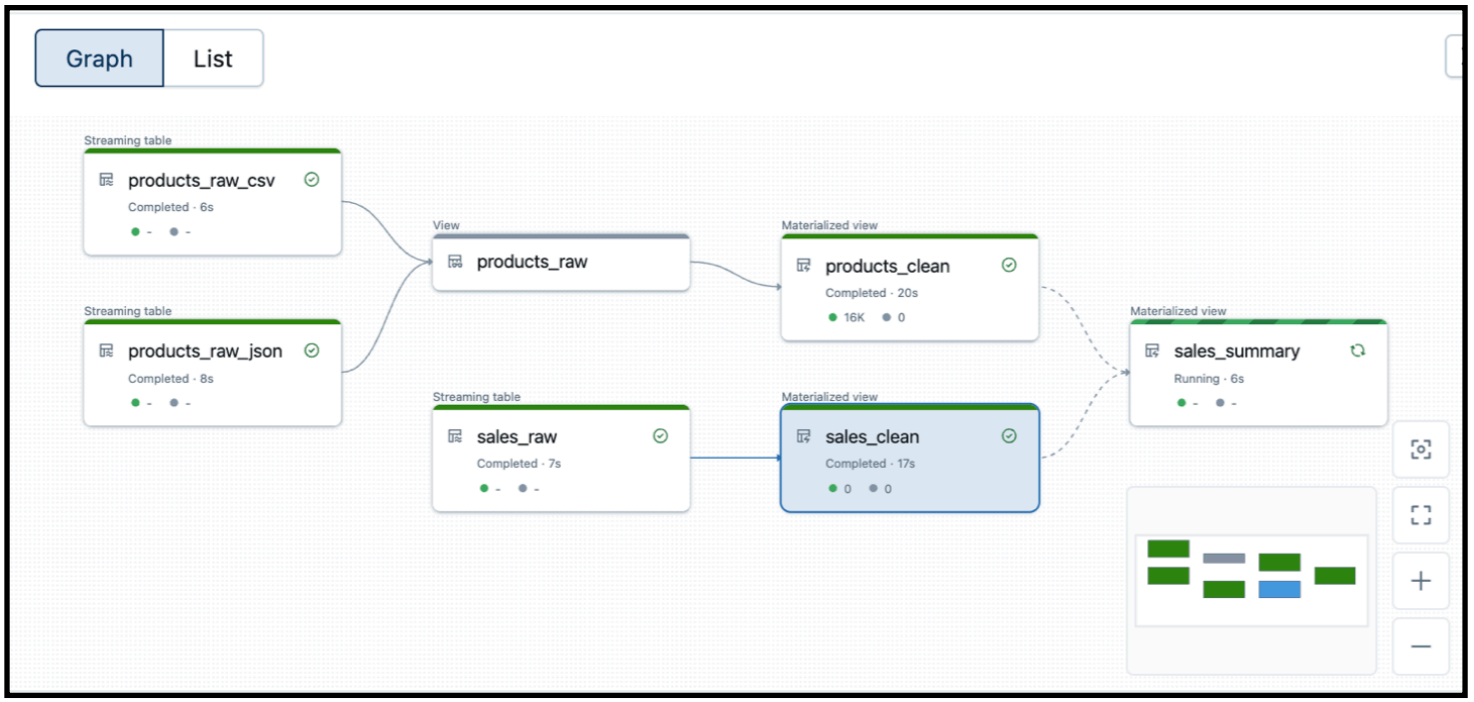
```
        .withColumn("sale_id", col("sale_id").cast("string"))
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("region", col("region").cast("string"))
        .withColumn("store_id", col("store_id").cast("string"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
        .na.drop()
    )

@dlt.table(
  name="products_clean",
  comment="Cleaned products data with schema evolution support",
  table_properties={"quality": "silver"}
)
def products_clean():
    return (
        dlt.read("products_raw")
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("product_name", col("product_name").cast("string"))
        .withColumn("category", col("category").cast("string"))
        .withColumn("price", col("price").cast("double"))
        .withColumn("discount_rate", col("discount_rate").cast("double"))
    )
```

C) Gold:

```
# Join sales + products and calculate KPIs
@dlt.table(
  name="sales_summary",
  comment="Daily sales summary by product and region",
  table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read("sales_clean")
    products = dlt.read("products_clean")

    return (
        sales.join(products, "product_id", "left")
```

```
        .groupBy("region", "sale_date", "product_id", "product_name",
"category")
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )
```

Healthy Pipeline:



Now creating a corrupted file:

sales_day_1

sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	sale_amt
1	157	APAC	Store_31	16	1289.03	2025-08-11 21:20:34	-1
2	65	APAC	Store_9	8	1301.48	2025-08-12 02:40:42	-2
3	416	LATAM	Store_25	19	3047.56	2025-08-10 00:09:00	-3
4	501	US	Store_29	18	1955.74	2025-07-28 09:49:56	-4
5	244	MEA	Store_6	20	2362.98	2025-08-15 00:28:03	-5
6	240	EU	Store_22	8	1516.96	2025-08-25 05:58:18	-6
7	854	EU	Store_24	10	178.92	2025-07-28 09:55:39	-7
8	69	US	Store_9	7	246.4	2025-08-10 22:41:18	-8
9	654	EU	Store_3	14	138.23	2025-08-02 05:50:22	-9
10	277	US	Store_8	15	145.13	2025-08-15 19:05:21	-10
11	584	LATAM	Store_4	13	2516.03	2025-08-08 13:20:55	-11
12	432	APAC	Store_1	13	241.39	2025-07-27 22:24:44	-12
13	797	US	Store_25	11	1914.19	2025-08-05 00:57:54	-13
14	64	MEA	Store_32	15	2545.81	2025-08-09 06:41:32	-14
15	256	US	Store_15	18	1482.75	2025-07-31 09:45:00	-15
16	924	LATAM	Store_14	7	670.92	2025-08-17 22:33:58	-16
17	245	US	Store_38	10	246.84	2025-07-31 15:43:06	
18	779	LATAM	Store_23	6	174.22	2025-07-29 16:52:39	Pvojfsvp
19	792	MEA	Store_13	7	435.92	2025-08-05 01:06:39	Cpovjpvboj
20	404	APAC	Store_8	10	1822.53	2025-08-05 03:28:00	
21	59	LATAM	Store_8	6	1133.17	2025-08-23 03:10:33	
22	867	MEA	Store_45	4	185.46	2025-08-17 19:47:57	
23	147	MEA	Store_50	18	3373.51	2025-07-30 07:52:49	
24	709	US	Store_46	13	432.09	2025-08-02 05:52:30	

Error in Pipeline:

Pipeline event log details

This event is associated with `pratham_cat_2608.pratham_schema.sales_raw` (Streaming table).

Summary JSON

WARN 2025-08-26 12:04:46 IST flow_progress

Flow 'pratham_cat_2608.pratham_schema.sales_raw' has encountered a schema change during execution and terminated. A new update using the new schema will be automatically started.

Error details

```
com.databricks.pipelines.common.errors.DLTSparkException: [FLOW_SCHEMA_CHANGED] Flow pratham_cat_2608.
pratham_schema.sales_raw has terminated since it encountered a schema change during execution.
The schema change is compatible with existing target schema and the next run of the flow can resume with
the new schema.
org.apache.spark.sql.streaming.StreamingQueryException: [STREAM_FAILED] Query [id = 48d72fe9-52bf-4600
-9743-30a1f3c9eef7, runId = 0f568aac-0455-4c32-83b5-e377c039ae7a] terminated with exception: [UNKNOWN_FIE
LD_EXCEPTION.NEW_FIELDS_IN_FILE] Encountered unknown fields during parsing: [sales_amt], which can be fix
ed by an automatic retry: true
 Unknown fields inside file path: abfss://raw@prathamstorage1.dfs.core.windows.net/sales/sales_day_6.csv
Rescued file schema: sales_amt STRING SQLSTATE: KD003 SQLSTATE: XXKST
=== Streaming Query ===
Identifier: pratham_cat_2608.pratham_schema.__materialization_mat_d401be79_fcf1_4d72_b799_c7a0da54dce8_sa
les_raw_1 [id = 48d72fe9-52bf-4600-9743-30a1f3c9eef7, runId = 0f568aac-0455-4c32-83b5-e377c039ae7a]
Current Start Offsets: {CloudFilesSource[abfss://raw@prathamstorage1.dfs.core.windows.net/sales/]: {"seqN
um":7,"sourceVersion":3,"lastBackfillStartTimeMs":1756186897484,"lastBackfillFinishTimeMs":175618690136
6,"lastInputPath":"abfss://raw@prathamstorage1.dfs.core.windows.net/sales/"}}
Current End Offsets: {CloudFilesSource[abfss://raw@prathamstorage1.dfs.core.windows.net/sales/]: {"seqNu
m":9,"sourceVersion":3,"lastBackfillStartTimeMs":1756186897484,"lastBackfillFinishTimeMs":175618690136
6,"lastInputPath":"abfss://raw@prathamstorage1.dfs.core.windows.net/sales/"}}
```

3 minutes ago (2m) 5 SQL

```
%sql
SELECT timestamp, level, message
FROM pratham_cat_2608.pratham_schema.event_log_pratham
WHERE level = 'ERROR'
ORDER BY timestamp DESC
LIMIT 20;
```

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [timestamp: timestamp, level: string ... 1 more field]

Table

	timestamp	level	message
1	2025-08-26T05:36:16.842+00:00	ERROR	Update 8be73e has failed. Failed to analyze flow 'pratham_cat_2608
2	2025-08-26T05:36:16.293+00:00	ERROR	Failed to resolve flow: 'pratham_cat_2608.pratham_schema.products

Automate the error Handling:

Code:

A) Bronze:

```
import dlt
from pyspark.sql.functions import *

# Ingest raw sales data (multiple daily CSV files in raw/sales/)
@dlt.table(
    name="sales_raw",
    comment="Raw sales data with automated error handling",
    table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .option("cloudFiles.schemaLocation", "/mnt/schema/sales/")  # schema tracking
        .option("cloudFiles.inferColumnTypes", "true")              # infer datatypes
        .option("cloudFiles.schemaEvolutionMode", "rescue")         # new/extra cols go to _rescued_data
        .load("abfss://raw@prathamstorage1.dfs.core.windows.net/sales/")
    )

# Products CSV
@dlt.table(
  name="products_raw_csv",
  comment="Raw product data from CSV",
  table_properties={"quality": "bronze"}
)
def products_raw_csv():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
```

```
        .option("header", "true")
        .load("abfss://raw@prathamstorage1.dfs.core.windows.net/
products/")
    )

# Products JSON
@dlt.table(
  name="products_raw_json",
  comment="Raw product data from JSON",
  table_properties={"quality": "bronze"}
)
def products_raw_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .load("abfss://raw@prathamstorage1.dfs.core.windows.net/
products/")
    )

# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
    return
dlt.read("products_raw_csv").unionByName(dlt.read("products_raw_json"),
allowMissingColumns=True)
```

B) Silver:

```
# ✅ Clean Sales Table
@dlt.table(
  name="sales_clean",
  comment="Cleaned sales data with enforced schema",
  table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
    df = dlt.read("sales_raw") \
        .withColumn("sale_id", col("sale_id").cast("string")) \
```

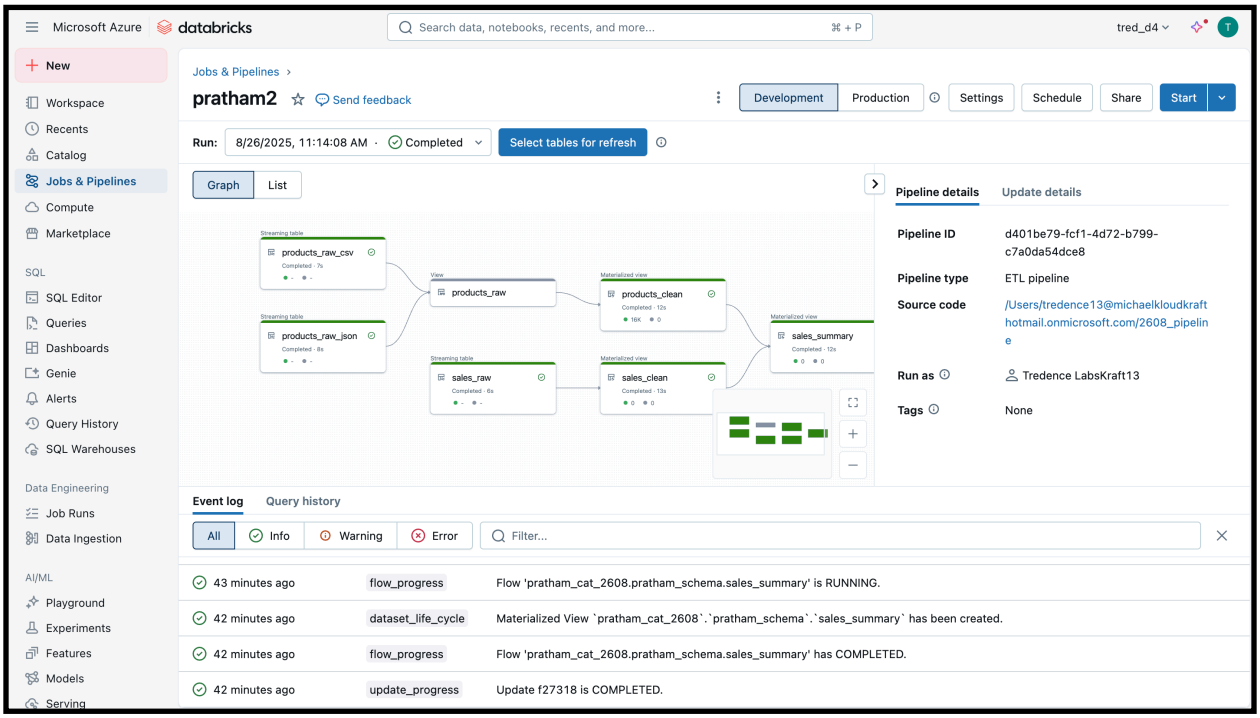
```
        .withColumn("product_id", col("product_id").cast("string")) \
        .withColumn("region", col("region").cast("string")) \
        .withColumn("store_id", col("store_id").cast("string")) \
        .withColumn("quantity", col("quantity").cast("int")) \
        .withColumn("sales_amount", col("sales_amount").cast("double")) \
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd")) \
        .select("sale_id", "product_id", "region", "store_id", "quantity",
"sales_amount", "sale_date")  # ✅ auto-drop unknowns
        .na.drop()

    return df
```

C) Gold:

```
# ✅ Clean Products Table
@dlt.table(
  name="products_clean",
  comment="Cleaned products data with schema evolution support",
  table_properties={"quality": "silver"}
)
def products_clean():
    df = dlt.read("products_raw_csv") \
        .withColumn("product_id", col("product_id").cast("string")) \
        .withColumn("product_name", col("product_name").cast("string")) \
        .withColumn("category", col("category").cast("string")) \
        .withColumn("price", col("price").cast("double")) \
        .withColumn("discount_rate", col("discount_rate").cast("double"))

    return df

# Join sales + products and calculate KPIs
@dlt.table(
  name="sales_summary",
  comment="Daily sales summary by product and region",
  table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read("sales_clean")
```

```
    products = dlt.read("products_clean")

    return (
        sales.join(products, "product_id", "left")
        .groupBy("region", "sale_date", "product_id", "product_name",
"category")
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )
```

Fixed running Pipeline:



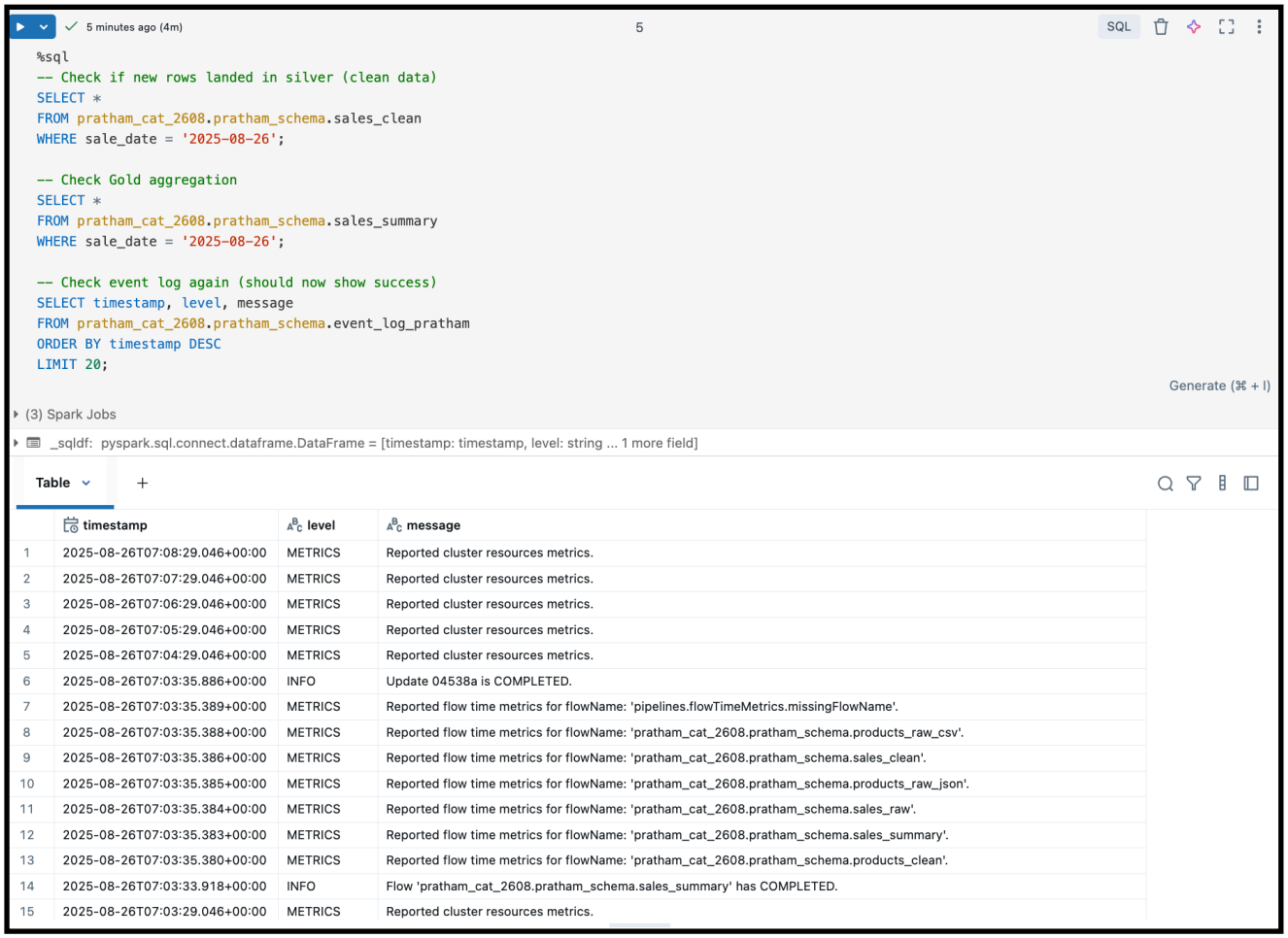
Pipeline event log details

This event is associated with `pratham_cat_2608.pratham_schema.sales_raw` (Streaming table).

Summary JSON

INFO 2025-08-26 12:32:49 IST flow_progress

Flow 'pratham_cat_2608.pratham_schema.sales_raw' has COMPLETED.



5 minutes ago (4m) 5 SQL

```
%sql
-- Check if new rows landed in silver (clean data)
SELECT *
FROM pratham_cat_2608.pratham_schema.sales_clean
WHERE sale_date = '2025-08-26';

-- Check Gold aggregation
SELECT *
FROM pratham_cat_2608.pratham_schema.sales_summary
WHERE sale_date = '2025-08-26';

-- Check event log again (should now show success)
SELECT timestamp, level, message
FROM pratham_cat_2608.pratham_schema.event_log_pratham
ORDER BY timestamp DESC
LIMIT 20;
```

Generate (⌘ + I)

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [timestamp: timestamp, level: string ... 1 more field]

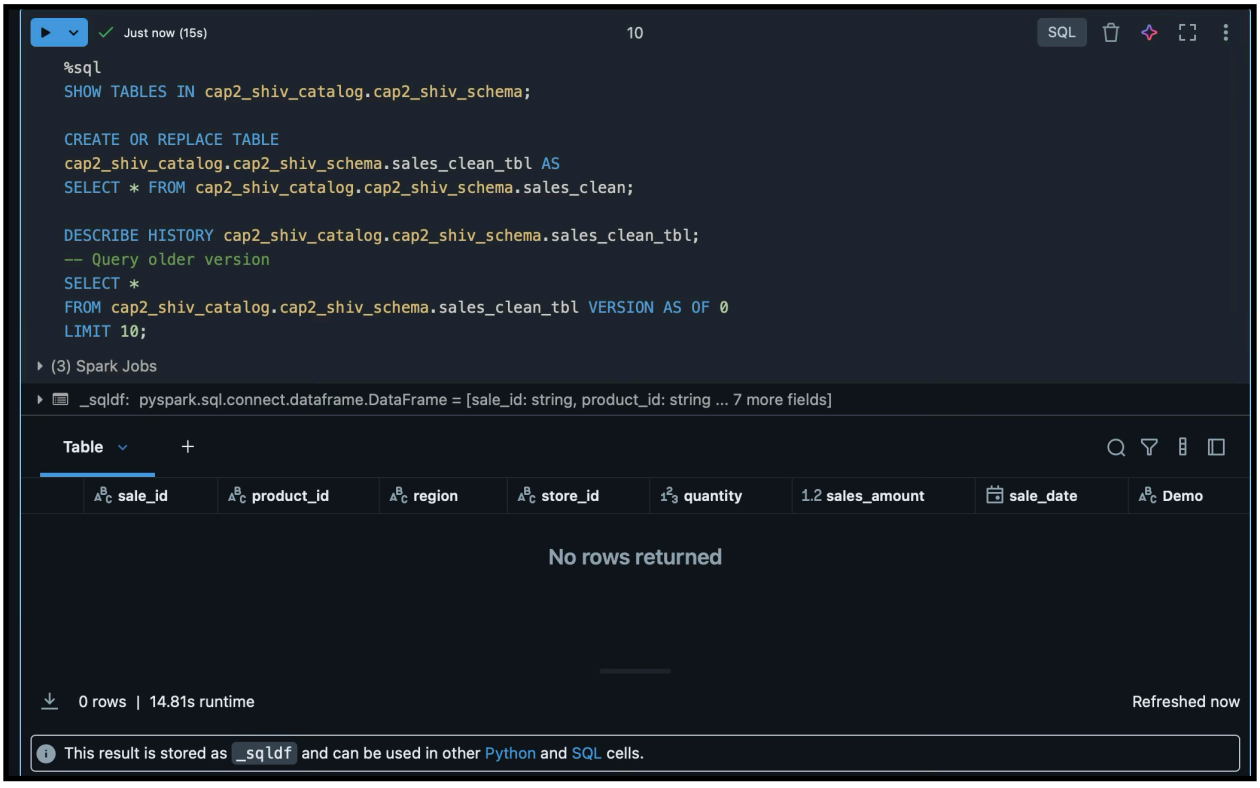
Table +

	timestamp	level	message
1	2025-08-26T07:08:29.046+00:00	METRICS	Reported cluster resources metrics.
2	2025-08-26T07:07:29.046+00:00	METRICS	Reported cluster resources metrics.
3	2025-08-26T07:06:29.046+00:00	METRICS	Reported cluster resources metrics.
4	2025-08-26T07:05:29.046+00:00	METRICS	Reported cluster resources metrics.
5	2025-08-26T07:04:29.046+00:00	METRICS	Reported cluster resources metrics.
6	2025-08-26T07:03:35.886+00:00	INFO	Update 04538a is COMPLETED.
7	2025-08-26T07:03:35.389+00:00	METRICS	Reported flow time metrics for flowName: 'pipelines.flowTimeMetrics.missingFlowName'.
8	2025-08-26T07:03:35.388+00:00	METRICS	Reported flow time metrics for flowName: 'pratham_cat_2608.pratham_schema.products_raw_csv'.
9	2025-08-26T07:03:35.386+00:00	METRICS	Reported flow time metrics for flowName: 'pratham_cat_2608.pratham_schema.sales_clean'.
10	2025-08-26T07:03:35.385+00:00	METRICS	Reported flow time metrics for flowName: 'pratham_cat_2608.pratham_schema.products_raw_json'.
11	2025-08-26T07:03:35.384+00:00	METRICS	Reported flow time metrics for flowName: 'pratham_cat_2608.pratham_schema.sales_raw'.
12	2025-08-26T07:03:35.383+00:00	METRICS	Reported flow time metrics for flowName: 'pratham_cat_2608.pratham_schema.sales_summary'.
13	2025-08-26T07:03:35.380+00:00	METRICS	Reported flow time metrics for flowName: 'pratham_cat_2608.pratham_schema.products_clean'.
14	2025-08-26T07:03:33.918+00:00	INFO	Flow 'pratham_cat_2608.pratham_schema.sales_summary' has COMPLETED.
15	2025-08-26T07:03:29.046+00:00	METRICS	Reported cluster resources metrics.

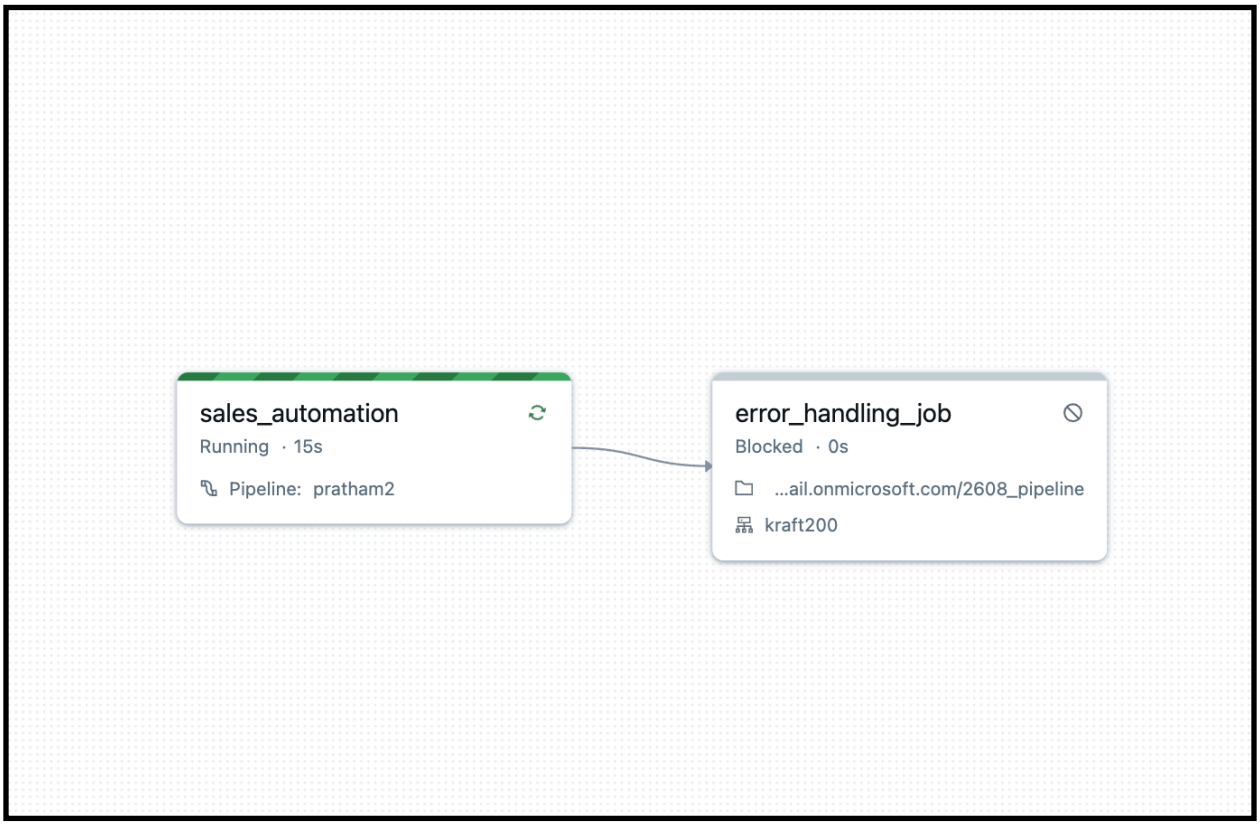
Time Travel :

Code:

```
%sql
SHOW TABLES IN pratham_cat_2608.pratham_schema;

%sql
CREATE OR REPLACE TABLE
pratham_cat_2608.pratham_schema.sales_clean_tbl AS
SELECT * FROM pratham_cat_2608.pratham_schema.sales_clean;

%sql
DESCRIBE HISTORY pratham_cat_2608.pratham_schema.sales_clean_tbl;
-- Query older version
SELECT *
FROM pratham_cat_2608.pratham_schema.sales_clean_tbl VERSION AS OF 1
LIMIT 10;
```



Creation of Automation Workflow Job:



I am getting an error in running the workflow job.

Visualization on Power BI: